Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»
Факультет программной инженерии и компьютерной техники

# Отчёт по Модулю 2

Классические методы машинного обучения

Линейная регрессия, $k$-NN, деревья решений,
логистическая регрессия

**Выполнили:** Ясаков Артём
Сарваров Тимур

**Группа:** P3313

**Проверил:** Жданов Андрей Дмитриевич

Санкт-Петербург, 2026 г.

# Contents

**Аннотация** **3**

**1 Введение** **4**

1.1 Цели модуля . . . . . 4
1.2 Критерии выбора метода . . . . . 4
1.3 Структура отчёта . . . . . 4

**2 Данные и общая методология** **5**

2.1 Выбранные датасеты . . . . . 5
2.2 Предобработка данных . . . . . 5
2.3 Инфраструктура воспроизводимости . . . . . 5

**3 Лабораторная работа 3. Линейная регрессия** **6**

3.1 Постановка задачи . . . . . 6
3.2 Описание данных . . . . . 6
3.3 Зачем метод . . . . . 7
3.4 Принцип работы . . . . . 7
3.5 Эксперименты . . . . . 8
3.6 Когда использовать . . . . . 10

**4 Лабораторная работа 4. Метод $k$-ближайших соседей** **11**

4.1 Постановка задачи . . . . . 11
4.2 Описание данных . . . . . 11
4.3 Зачем метод . . . . . 11
4.4 Принцип работы . . . . . 12
4.5 Эксперименты . . . . . 12
4.6 Когда использовать . . . . . 13

**5 Лабораторная работа 5. Деревья решений** **14**

5.1 Постановка задачи . . . . . 14
5.2 Описание данных . . . . . 14
5.3 Зачем метод . . . . . 14
5.4 Принцип работы . . . . . 15
5.5 Эксперименты . . . . . 15
5.6 Когда использовать . . . . . 17

**6 Лабораторная работа 6. Логистическая регрессия** **18**

6.1 Постановка задачи . . . . . 18
6.2 Описание данных . . . . . 18
6.3 Зачем метод . . . . . 19

6.4 Принцип работы . . . . . 19
6.5 Эксперименты . . . . . 20
6.6 Финальная модель . . . . . 22
6.7 Когда использовать . . . . . 22

**7 Сравнение методов** **23**
7.1 Сводная таблица . . . . . 23
7.2 Рекомендации «если. . . , то. . . » . . . . . 23
7.3 Итоговые метрики . . . . . 23

**8 Заключение** **24**

**Использование искусственного интеллекта** **25**

# Аннотация

В рамках Модуля 2 дисциплины «Системы искусственного интеллекта» реализованы «с нуля» четыре классических алгоритма машинного обучения: линейная регрессия (California Housing), метод $k$-ближайших соседей (Wine), дерево решений ID3 (Mushroom) и логистическая регрессия (Titanic). Все реализации выполнены на Python с использованием только NumPy и Pandas (кроме дерева решений, использующего исключительно стандартную библиотеку). Проведён сравнительный анализ методов по метрикам качества, исследовано влияние гиперпараметров и наборов признаков. Сформулированы практические рекомендации по выбору метода в зависимости от типа задачи, размера данных и требований к интерпретируемости.

**Ключевые слова:** машинное обучение, линейная регрессия, $k$-ближайших соседей, дерево решений, логистическая регрессия, градиентный спуск, метод Ньютона, классификация, регрессия, гиперпараметры.

# 1 Введение

## 1.1 Цели модуля

Модуль 2 направлен на освоение фундаментальных алгоритмов машинного обучения через их самостоятельную реализацию. В ходе модуля решаются следующие задачи:

- Линейная регрессия — предсказание непрерывной величины (стоимость жилья) по набору числовых и категориальных признаков.
- $k$-ближайших соседей — классификация объектов (сорт вина) без явного обучения модели.
- Дерево решений — интерпретируемая классификация (съедобность грибов) с автоматическим построением правил.
- Логистическая регрессия — бинарная классификация (выживаемость пассажиров Титаника) с вероятностной интерпретацией.

## 1.2 Критерии выбора метода

Выбор алгоритма определяется несколькими ключевыми факторами:

- **Тип целевой переменной.** Непрерывная — регрессия; категориальная — классификация.
- **Размер и качество данных.** $k$-NN чувствителен к размерности и требует масштабирования; деревья решений устойчивы к пропускам и выбросам.
- **Интерпретируемость.** Линейные модели и деревья позволяют понять вклад каждого признака; $k$-NN не даёт такой возможности.
- **Стоимость ошибок.** В задачах классификации выбор между Precision и Recall зависит от цены ложноположительных и ложноотрицательных срабатываний.

## 1.3 Структура отчёта

Отчёт последовательно описывает четыре лабораторные работы, каждая из которых содержит: постановку задачи, описание данных, принцип работы метода с псевдокодом, результаты экспериментов и рекомендации по применению. Завершается отчёт сравнительным анализом методов и общими выводами.

# 2 Данные и общая методология

## 2.1 Выбранные датасеты

Для каждой лабораторной работы использован отдельный датасет, подобранный под специфику метода (таблица 1).

Table 1: Обзор датасетов

| Лаб. | Метод | Датасет | Объектов | Признаков | Классов |
|---|---|---|---|---|---|
| 3 | Линейная регрессия | California Housing | 20 640 | 8 + 1 кат. | — |
| 4 | $k$-NN | Wine | 178 | 13 | 3 |
| 5 | Дерево решений | Mushroom | 8 124 | 22 кат. | 2 |
| 6 | Логист. регрессия | Titanic | 891 | 7 + 2 кат. | 2 |

## 2.2 Предобработка данных

Во всех работах применялась единая методология:

1. **Пропуски:** заполнение медианой (числовые) или исключение признака (категориальные с большим числом пропусков, например, `deck` в Titanic — 688 из 891).
2. **Категориальные признаки:** one-hot кодирование с `drop_first` для линейных моделей; для дерева решений используются исходные категории.
3. **Масштабирование:** Z-нормировка по статистикам обучающей выборки для методов, чувствительных к масштабу ($k$-NN, логистическая регрессия).
4. **Разбиение:** train/test 80/20 со стратификацией (для классификации) или случайное (для регрессии).
5. **Random seed:** 42 — фиксирован для воспроизводимости.

## 2.3 Инфраструктура воспроизводимости

- Язык: Python 3.13.
- Зависимости: NumPy, Pandas, Matplotlib, Seaborn, scikit-learn (только для `train_test_split` и `StandardScaler` в лабах 4 и 6; дерево решений — исключительно стандартная библиотека).
- Запуск: `python3 lab_*.py` из соответствующей директории.

# 3 Лабораторная работа 3. Линейная регрессия

## 3.1 Постановка задачи

Предсказать медианную стоимость жилья (`median_house_value`) в районах Калифорнии по набору социально-экономических и географических признаков.

## 3.2 Описание данных

- **Источник:** California Housing Prices (A. Géron, Hands-On Machine Learning).
- **Объектов:** 20 640 районов.
- **Признаки:** 8 числовых (`longitude`, `latitude`, `housing_median_age`, `total_rooms`, `total_bedrooms`, `population`, `households`, `median_income`) + 1 категориальный (`ocean_proximit` 5 значений).
- **Целевая:** `median_house_value` (доллары США).
- **Пропуски:** 207 в `total_bedrooms` (заполнены медианой 435.0).

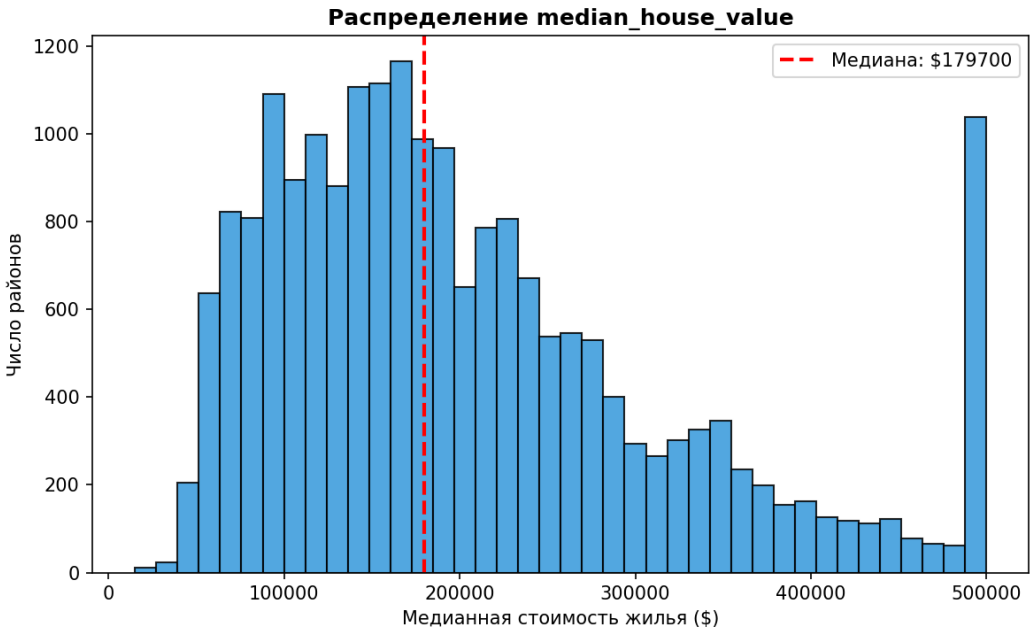



Figure 1: Распределение целевой переменной

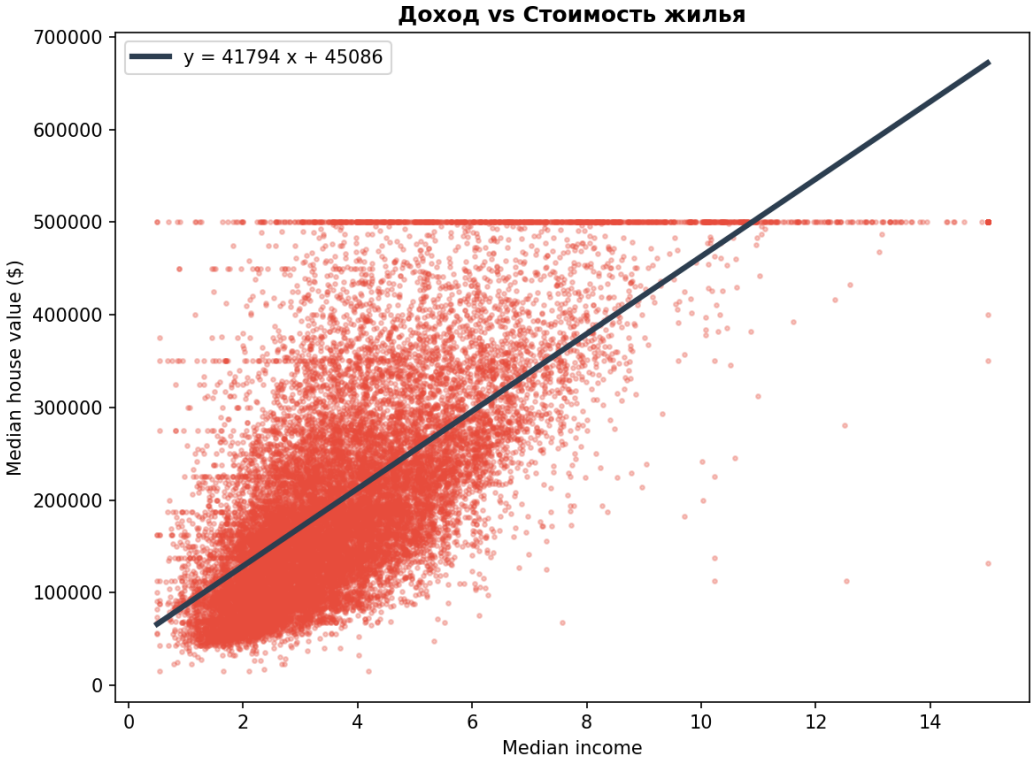



Figure 2: Доход vs Стоимость жилья

## 3.3 Зачем метод

Линейная регрессия — базовый метод для задач с непрерывной целевой переменной. Модель предполагает линейную зависимость: $\hat{y} = \beta_0 + \sum_{j=1}^{d} \beta_j x_j$.

**Условия применимости:** линейная связь признаков с целевой, отсутствие сильной мультиколлинеарности, нормальность остатков. **Ограничения:** не улавливает нелинейные зависимости, чувствительна к выбросам.

## 3.4 Принцип работы

Метод наименьших квадратов (МНК) минимизирует сумму квадратов остатков: $\hat{\beta} = \arg\min_\beta \|y - X\beta\|_2^2$. Аналитическое решение через нормальные уравнения: $\hat{\beta} = (X^T X)^{-1} X^T y$.

**Псевдокод:** Линейная регрессия (МНК)

1. **Вход:** $X \in \mathbb{R}^{n \times d}$, $y \in \mathbb{R}^n$.
2. Добавить столбец единиц: $X_{\text{aug}} = [\mathbf{1} \mid X]$.
3. Составить нормальные уравнения: $A = X_{\text{aug}}^T X_{\text{aug}}$, $b = X_{\text{aug}}^T y$.
4. Решить $A\beta = b$ методом Гаусса с выбором главного элемента.
5. **Выход:** $\beta = [\beta_0, \beta_1, \ldots, \beta_d]$.

**Ключевой фрагмент — решение СЛАУ методом Гаусса (без `np.linalg`):**

Listing 1: Метод Гаусса с выбором главного элемента

```
def gaussian_solve(A, b):
```

```
    A, b = np.array(A, dtype=float, copy=True), np.array(b, dtype=float,
        copy=True)
    n = len(b)
    for k in range(n):
        pivot = k + int(np.argmax(np.abs(A[k:, k])))
        if pivot != k: A[[k, pivot]] = A[[pivot, k]]; b[[k, pivot]] = b[[
            pivot, k]]
        A[k, k:] /= A[k, k]; b[k] /= A[k, k]
        for i in range(n):
            if i == k: continue
            factor = A[i, k]; A[i, k:] -= factor * A[k, k:]; b[i] -=
                factor * b[k]
    return b
```

## 3.5 Эксперименты

Построены 4 модели с последовательным расширением набора признаков:

1. **Модель 1** — только `median_income`.
2. **Модель 2** — `median_income`, `housing_median_age`, `latitude`, `longitude`.
3. **Модель 3** — все 8 числовых признаков + one-hot кодирование `ocean_proximity` (4 бинарных признака, базовая категория: `<1H OCEAN`).
4. **Бонус** — модель 3 + синтетический признак `bedrooms_per_room`.

Table 2: Результаты линейной регрессии

| Модель | Признаков | $R^2$ (train) | $R^2$ (test) |
|---|---|---|---|
| Модель 1 (только доход) | 1 | 0.4748 | 0.4683 |
| Модель 2 (+гео+возраст) | 4 | 0.5954 | 0.5886 |
| Модель 3 (все признаки) | 12 | 0.6462 | 0.6423 |
| Бонус (+синтетический) | 13 | 0.6487 | **0.6446** |

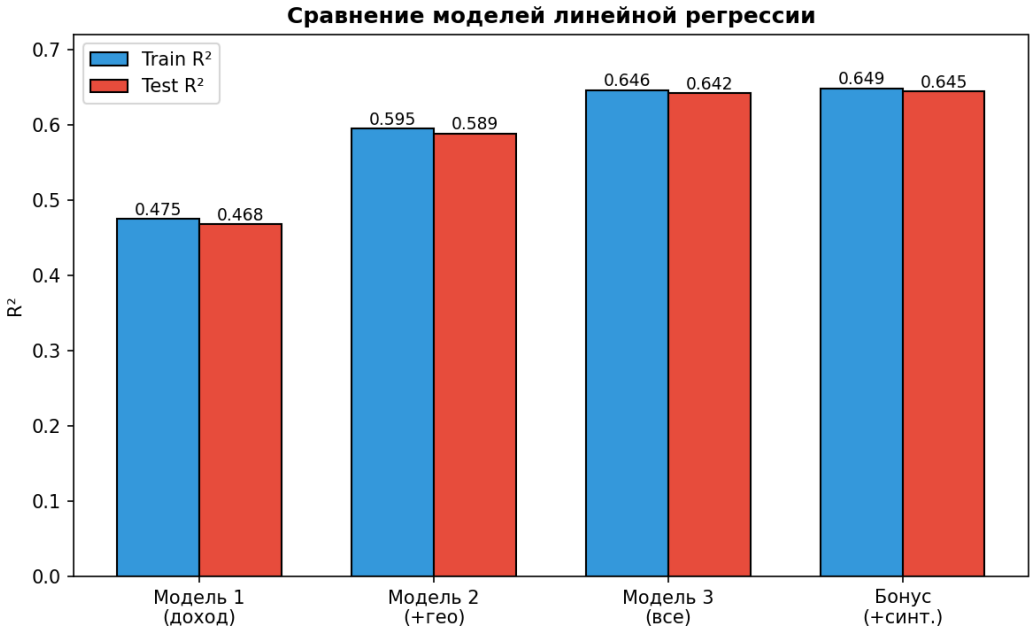



Figure 3: Сравнение $R^2$ моделей

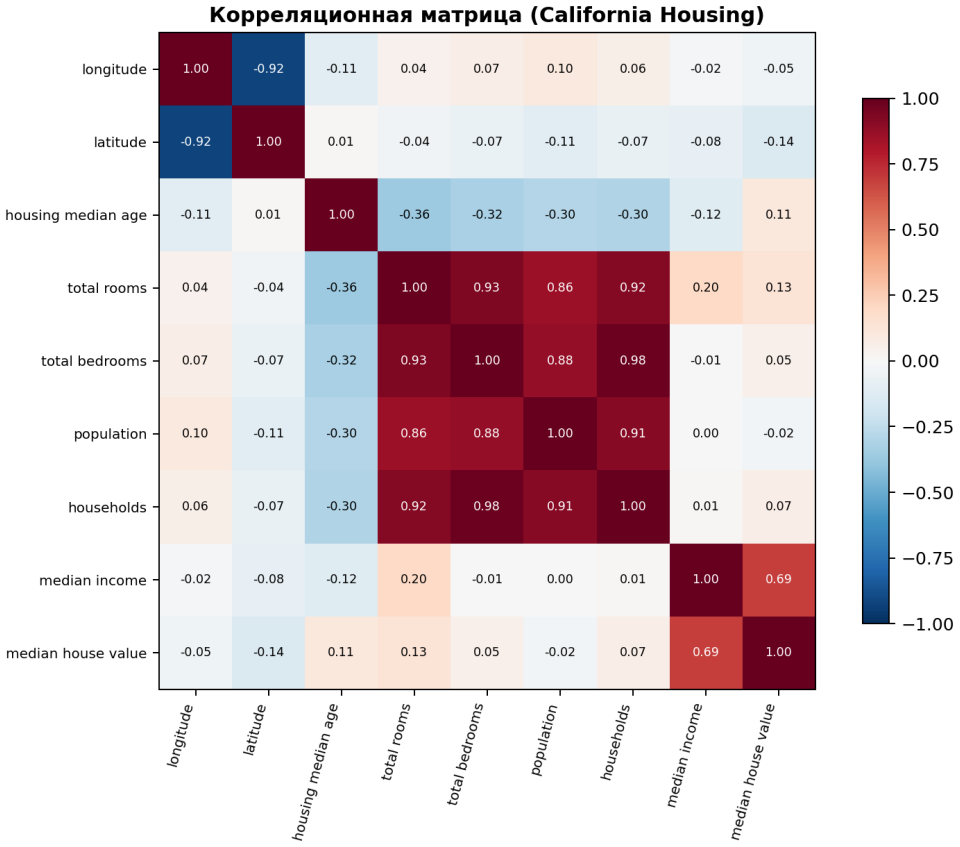



Figure 4: Корреляционная матрица числовых признаков

**Анализ:**

- `median_income` — доминирующий признак: одна лишь эта переменная даёт $R^2 \approx 0.47$.
- Добавление географических координат и возраста жилья увеличивает $R^2$ на $\sim 0.12$.
- Полный набор с категориальными признаками даёт $R^2 \approx 0.64$; синтетический признак даёт минимальный прирост (+0.002).

- Переобучение отсутствует: разница train/test $R^2$ минимальна во всех моделях.

## 3.6 Когда использовать

- Прогнозирование цен недвижимости, аренды.
- Оценка влияния факторов на экономические показатели.
- Быстрый baseline для задач регрессии перед применением сложных моделей.
- Задачи, где требуется интерпретация коэффициентов (на сколько изменится целевая при изменении признака на единицу).

# 4 Лабораторная работа 4. Метод $k$-ближайших соседей

## 4.1 Постановка задачи

Классифицировать сорт вина по 13 химическим показателям. Многоклассовая задача (3 сорта винограда).

## 4.2 Описание данных

- **Источник:** `sklearn.datasets.load_wine` (UCI Wine).
- **Объектов:** 178 образцов (сбалансированы: 59, 71, 48).
- **Признаки:** 13 числовых (alcohol, malic_acid, ash, alcalinity_of_ash, magnesium, total_phenols, flavanoids, nonflavanoid_phenols, proanthocyanins, color_intensity, hue, od280/od315, proline).
- **Пропуски:** отсутствуют.

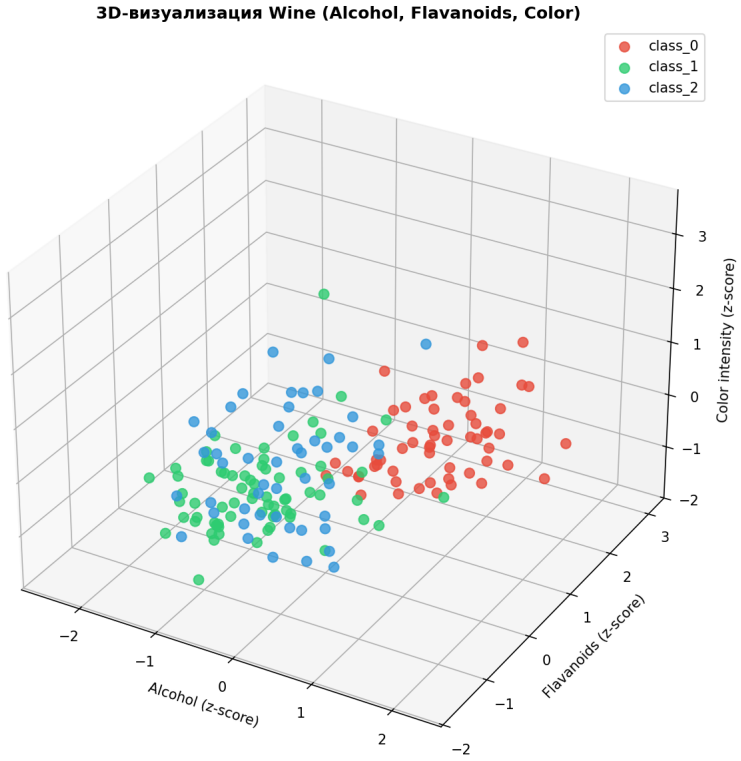



Figure 5: 3D-визуализация: alcohol, flavanoids, color_intensity

## 4.3 Зачем метод

$k$-NN — метод «ленивого обучения»: модель не имеет явной фазы обучения, а запоминает все обучающие примеры. Классификация выполняется по принципу «похожие объекты принадлежат одному классу».

**Требования:** обязательное масштабирование признаков (чувствителен к масштабу), неэффективен при высокой размерности («проклятие размерности»). **Ограничения:**

медленный на этапе предсказания ($O(n_{\text{train}} \cdot d)$ на один объект), чувствителен к шуму и неинформативным признакам.

## 4.4 Принцип работы

**Псевдокод:** $k$-ближайших соседей

1. **Вход:** обучающая выборка $(X_{\text{train}}, y_{\text{train}})$, тестовый объект $x$, число соседей $k$.
2. Вычислить евклидовы расстояния до всех обучающих объектов: $d_i = \|x - x_i\|_2$.
3. Найти $k$ объектов с наименьшими $d_i$.
4. Класс — мажоритарное голосование: $\hat{y} = \arg\max_c \sum_{i \in \text{top-}k} [y_i = c]$.
5. **Выход:** предсказанный класс $\hat{y}$.

Для ускорения расстояния вычисляются векторизованно через тождество: $\|a-b\|^2 = \|a\|^2 + \|b\|^2 - 2a \cdot b$, что позволяет вычислить всю матрицу расстояний $(n_{\text{test}} \times n_{\text{train}})$ за одно матричное умножение.

Listing 2: Векторизованное вычисление расстояний и голосование

```
#                                          (n_test x n_train)

dists = (np.sum(X_test**2, axis=1)[:, None]
       + np.sum(X_train**2, axis=1)[None, :]
       - 2 * np.dot(X_test, X_train.T))
#
top_k = y_train[np.argsort(dists[i])[:k]]
pred = np.argmax(np.bincount(top_k, minlength=n_classes))
```

## 4.5 Эксперименты

Построены две модели с разными наборами признаков:

1. **Модель 1 (случайные):** 6 случайно отобранных признаков (`od280/od315`, `color_intensity`, `alcohol`, `proanthocyanins`, `total_phenols`, `ash`).
2. **Модель 2 (фиксированные):** 5 наиболее информативных признаков (`alcohol`, `flavanoids`, `color_intensity`, `proline`, `total_phenols`).

Исследованы $k \in \{1, 3, 5, 7, 10, 15\}$.

Table 3: Точность $k$-NN при разных $k$

| $k$ | Модель 1 (случайные) | Модель 2 (фиксированные) |
|---|---|---|
| 1 | 0.8889 | 0.9630 |
| 3 | 0.8889 | 0.9630 |
| 5 | 0.9074 | 0.9630 |
| 7 | **0.9259** | 0.9630 |
| 10 | 0.8889 | **0.9815** |
| 15 | 0.8889 | 0.9444 |

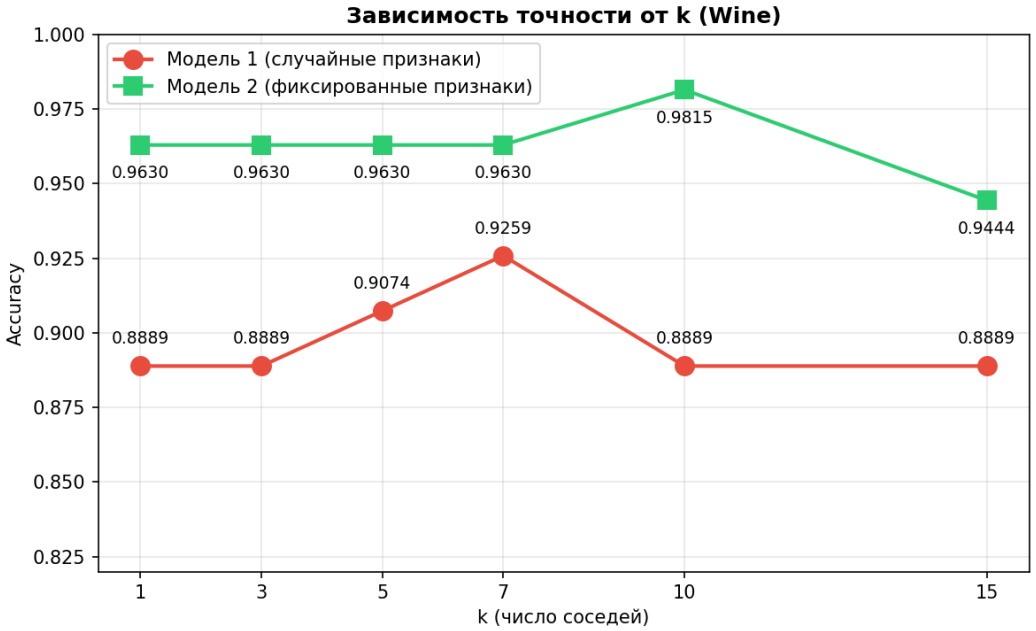



Figure 6: Зависимость точности от $k$

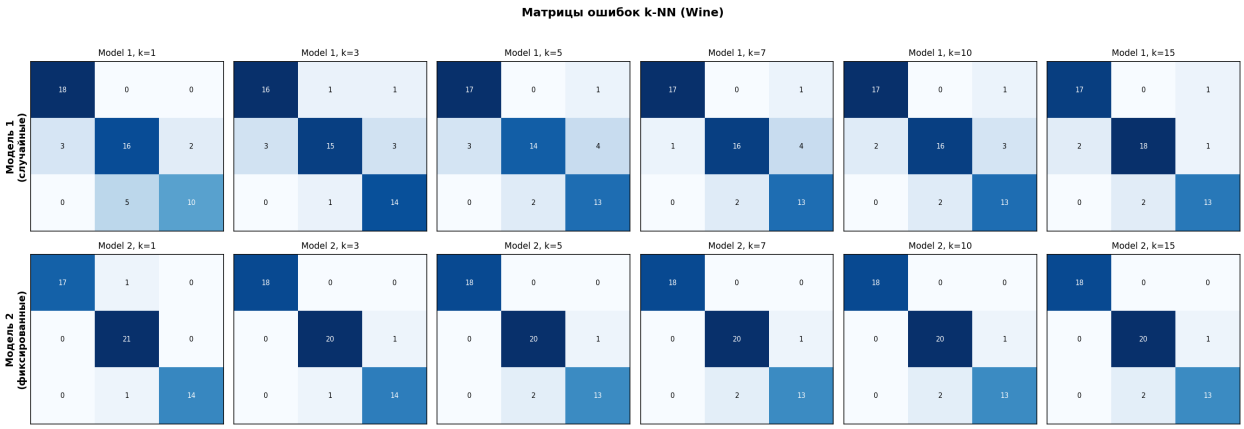



Figure 7: Матрицы ошибок (верхний ряд — Модель 1, нижний — Модель 2)

**Анализ:**

- Модель 2 стабильно превосходит Модель 1 за счёт включения `flavanoids` и `proline` — наиболее разделяющих признаков.
- Оптимальное $k = 10$ для Модели 2 (Accuracy = 0.9815).
- При $k = 15$ точность снижается из-за «размытия» границ решающей области.
- Класс 0 распознаётся практически безошибочно; ошибки сосредоточены между классами 1 и 2 (матрицы ошибок подтверждают).

## 4.6 Когда использовать

- Быстрый baseline без настройки (нет фазы обучения).
- Многоклассовая классификация с небольшим числом признаков.
- Рекомендательные системы (поиск похожих товаров/фильмов).
- Обнаружение аномалий (по расстоянию до соседей).

# 5 Лабораторная работа 5. Деревья решений

## 5.1 Постановка задачи

Классифицировать грибы на съедобные (`e`) и ядовитые (`p`) по набору категориальных признаков. Задача бинарной классификации с высокими требованиями к интерпретируемости (цена ошибки — жизнь человека).

## 5.2 Описание данных

- **Источник:** UCI Mushroom (Agaricus-Lepiota).
- **Объектов:** 8 124 образца (4 208 съедобных, 3 916 ядовитых).
- **Признаки:** 22 категориальных (cap-shape, cap-color, odor, gill-size, spore-print-color, population, habitat и др.).
- **Пропуски:** 2 480 в `stalk-root` (обрабатываются как отдельная категория `?`).

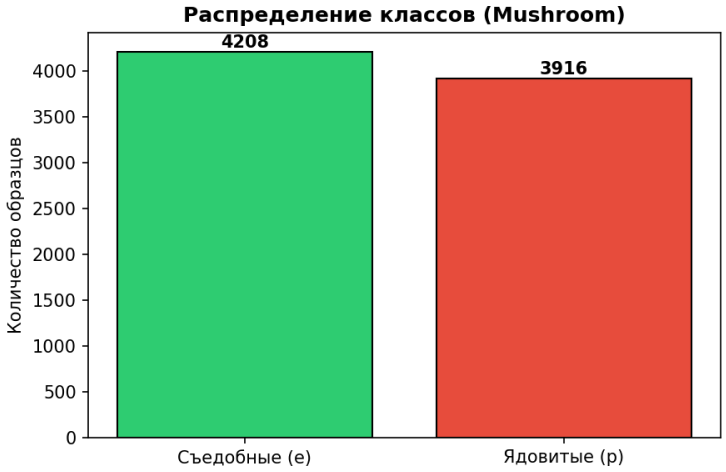



Figure 8: Распределение классов

## 5.3 Зачем метод

Деревья решений привлекательны своей **интерпретируемостью**: модель представляет собой набор правил «если… то… », понятных человеку. Метод автоматически выявляет нелинейные взаимодействия, не требует масштабирования и устойчив к выбросам.

**Ограничения:** склонность к переобучению (глубокие деревья), чувствительность к малым изменениям данных, жадная стратегия построения не гарантирует глобального оптимума.

## 5.4 Принцип работы

Алгоритм ID3 строит дерево рекурсивно, выбирая на каждом шаге признак, дающий максимальный прирост информации:

$$IG(S, A) = H(S) - \sum_{v \in \text{values}(A)} \frac{|S_v|}{|S|} H(S_v),$$

где $H(S) = -\sum_c p_c \log_2 p_c$ — энтропия Шеннона.

**Псевдокод:** Построение дерева (ID3)

1. **Вход:** выборка $S$, список признаков $F$.
2. Если все объекты одного класса или $F$ пуст — создать лист с вероятностью класса $P(\text{positive})$.
3. Для каждого признака $A \in F$ вычислить $\text{IG}(S, A)$.
4. Выбрать $A^* = \arg\max_A \text{IG}(S, A)$.
5. Для каждого значения $v \in A^*$ рекурсивно построить поддерево на $S_v$ с признаками $F \setminus \{A^*\}$.
6. **Выход:** узел с признаком $A^*$ и дочерними поддеревьями.

Из 22 признаков случайно отбираются $\lfloor\sqrt{22}\rfloor = 4$ (аналог Random Forest). Библиотеки: только `csv`, `math`, `random` — никаких NumPy/Pandas/sklearn.

Listing 3: Построение дерева ID3 — выбор лучшего признака

```
def build_tree(rows, features):
    prob = sum(r['class'] == 'p' for r in rows) / len(rows)
    if prob in (0, 1) or not features:
        return Node(len(rows), prob)
    best_gain, best_feature = -math.inf, None
    for f in features:
        groups = {}; [groups.setdefault(r[f], []).append(r) for r in rows]
        gain = entropy(rows) - sum(
            len(g)/len(rows)*entropy(g) for g in groups.values())
        if gain > best_gain + 1e-12:
            best_gain, best_feature, best_groups = gain, f, groups
    node = Node(len(rows), prob); node.feature = best_feature
    node.children = {v: build_tree(g, [x for x in features if x !=
       best_feature])
                     for v, g in sorted(best_groups.items())}
    return node
```

## 5.5 Эксперименты

Отобранные признаки: `population`, `bruises`, `cap-shape`, `gill-color`. Корень дерева — `gill-color` (цвет пластинок гриба).

Table 4: Характеристики и метрики дерева

| Параметр | Значение | Метрика (test) | Значение |
|---|---|---|---|
| Узлов / листьев | 141 / 96 | Accuracy | 0.9335 |
| Глубина | 4 | Precision | 0.9592 |
| Корень | gill-color | Recall | 0.9004 |
| Макс. потомков | 12 | ROC-AUC | 0.9828 |

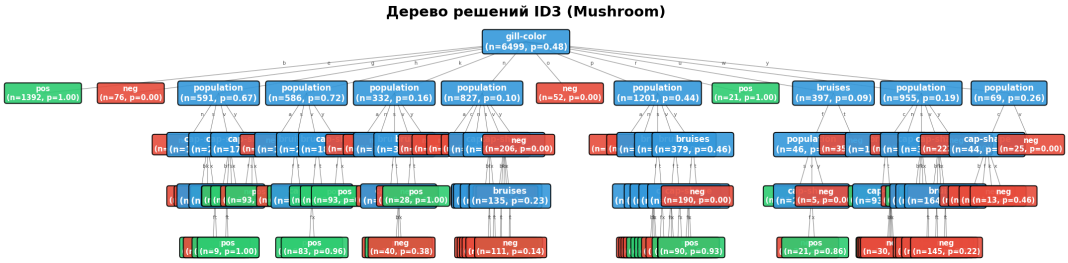



Figure 9: Визуализация построенного дерева (первые уровни)

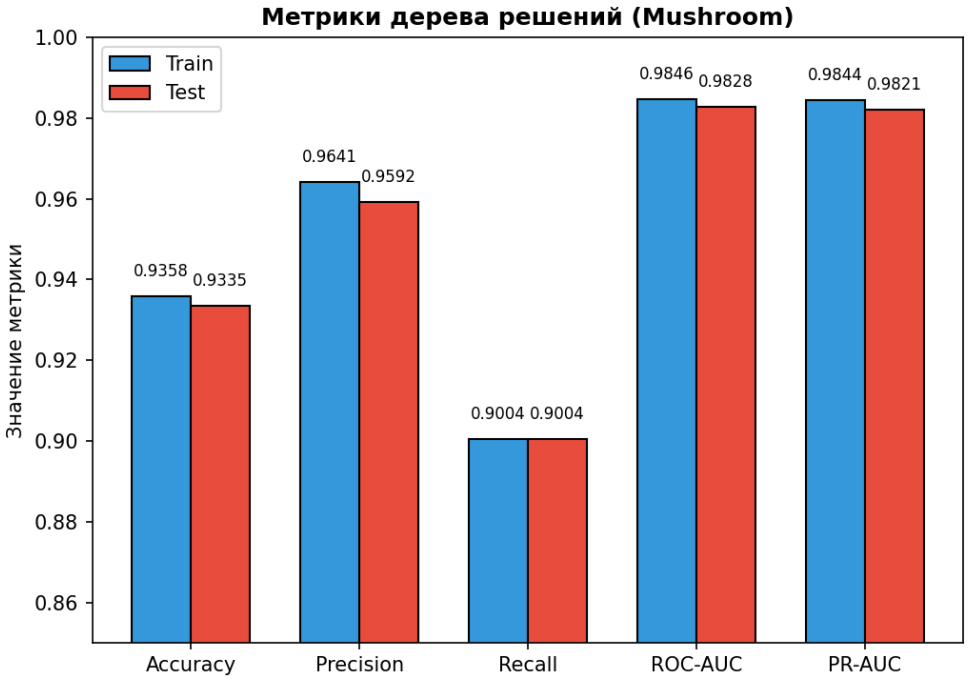



Figure 10: Сравнение метрик train/test

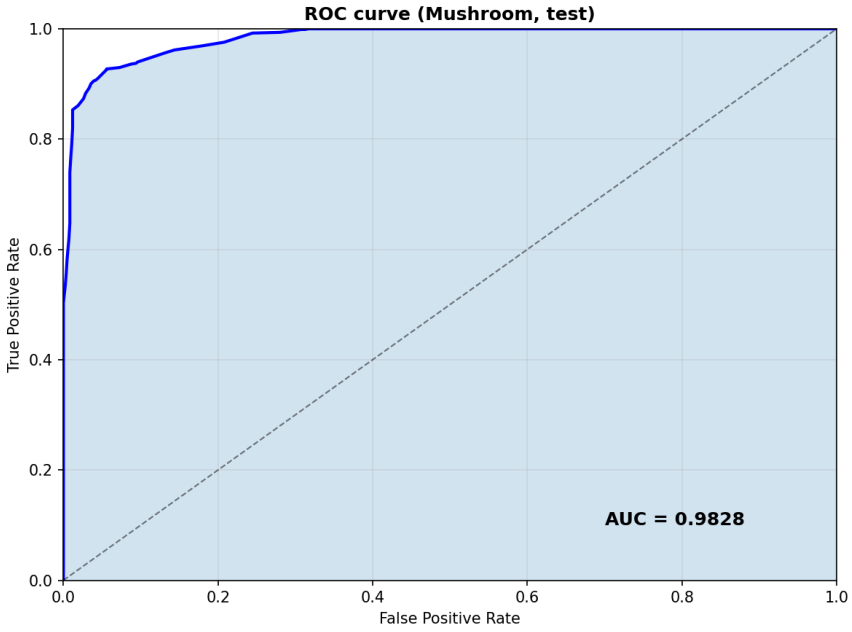



Figure 11: ROC-кривая на тестовой выборке

**Анализ:**

- Дерево глубиной 4 с 141 узлом — компактная и интерпретируемая модель. Ограничение глубины выступает как неявная регуляризация (прунинг).
- Наиболее информативный признак — `gill-color` (цвет пластинок гриба).
- Precision 0.96, Recall 0.90 — модель осторожна: предпочитает пропустить съедобный гриб, чем ошибочно назвать ядовитый съедобным (что соответствует цене ошибки).
- ROC-AUC = 0.983 — отличная разделяющая способность.
- Разница train/test минимальна — переобучения нет.

## 5.6 Когда использовать

- Медицинская диагностика (понятные правила).
- Кредитный скоринг (требуется объяснение решения).
- Задачи с категориальными признаками и пропусками.
- Предварительный анализ данных (feature importance).

# 6 Лабораторная работа 6. Логистическая регрессия

## 6.1 Постановка задачи

Предсказать, выжил ли пассажир Титаника (бинарная классификация), по набору признаков: класс билета, пол, возраст, количество родственников, стоимость билета, порт посадки.

## 6.2 Описание данных

- **Источник:** `seaborn.load_dataset('titanic')`.
- **Объектов:** 891 пассажир (549 погибших / 342 выживших; классы не сбалансированы — 61.6% / 38.4%).
- **Признаки после обработки:** 9 числовых: sex, age, sibsp, parch, fare, embarked_Q/S, pclass_2/3.
- **Пропуски:** age (177), embarked (2) — заполнены медианой.

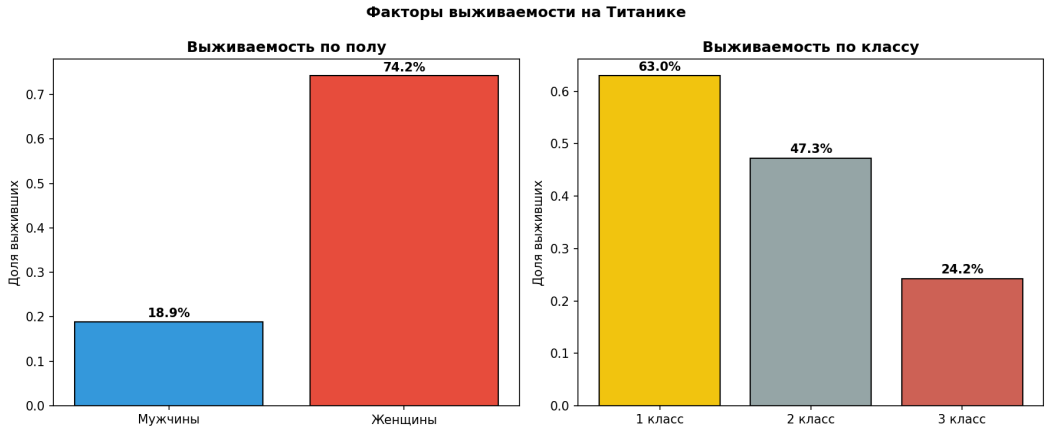



Figure 12: Факторы выживаемости

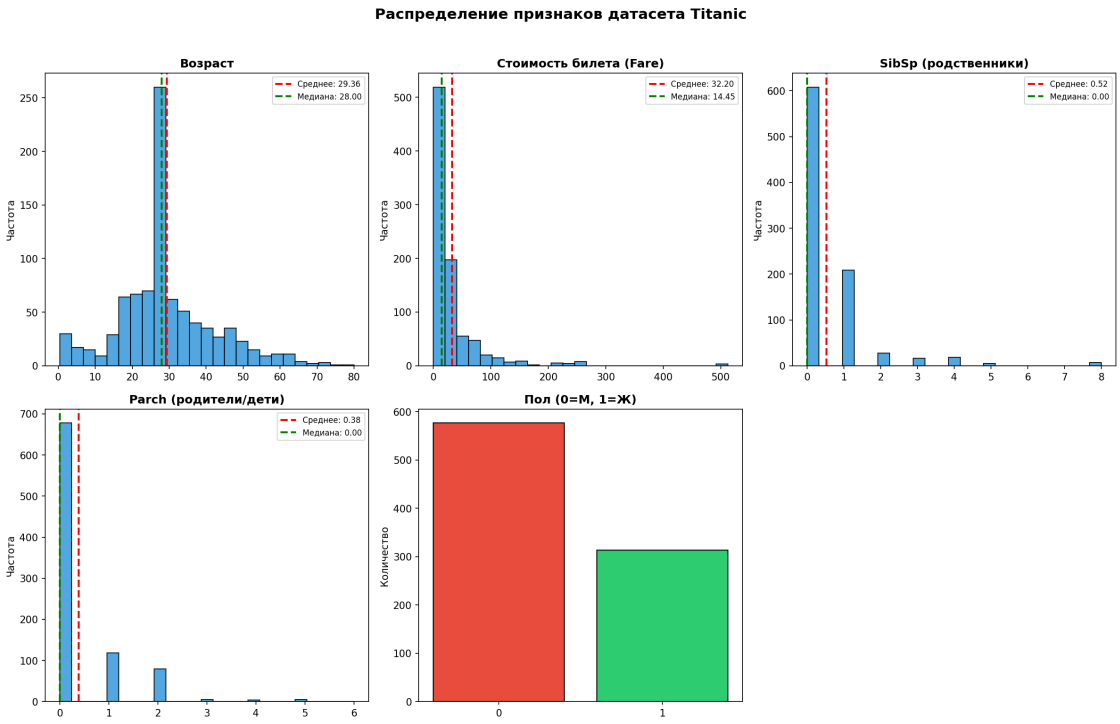



Figure 13: Распределение признаков датасета Titanic

## 6.3 Зачем метод

Логистическая регрессия — основной метод бинарной классификации, дающий вероятностную интерпретацию. В отличие от линейной регрессии, предсказывает вероятность принадлежности к классу через сигмоиду:

$$P(y = 1 \mid x) = \sigma(w^T x + b) = \frac{1}{1 + e^{-(w^T x + b)}}.$$

**Интерпретация коэффициентов:** $\beta_j$ показывает изменение логарифма шансов ($\log \frac{P}{1-P}$) при изменении $x_j$ на единицу. **Ограничения:** предполагает линейную разделимость в логит-пространстве, чувствительна к масштабу признаков.

## 6.4 Принцип работы

Обучение сводится к минимизации log loss:

$$\mathcal{L}(w, b) = -\frac{1}{n} \sum_{i=1}^{n} \big[ y_i \ln \hat{y}_i + (1 - y_i) \ln(1 - \hat{y}_i) \big],$$

где $\hat{y}_i = \sigma(w^T x_i + b)$.

**Псевдокод:** Логистическая регрессия (градиентный спуск)

1. **Вход:** $X_{\text{train}}$, $y_{\text{train}}$, $\eta$, $N$.
2. Инициализировать $w = 0$, $b = 0$.
3. **Повторить** $N$ раз: $\hat{y} = \sigma(Xw + b)$; $\nabla_w = \frac{1}{n} X^T(\hat{y} - y)$; $\nabla_b = \frac{1}{n}\sum(\hat{y}_i - y_i)$; $w := w - \eta\nabla_w$; $b := b - \eta\nabla_b$.
4. **Выход:** $w, b$.

Реализован также **метод Ньютона**: $\theta := \theta - H^{-1}\nabla\mathcal{L}(\theta)$, где $H = \frac{1}{n} X_{\text{aug}}^T S X_{\text{aug}} + \lambda I$ — регуляризованная матрица Гессе, $S = \text{diag}(\hat{y}_i(1 - \hat{y}_i))$.

Listing 4: Градиентный спуск и метод Ньютона

```
#
class LogisticRegressionGD:
    def fit(self, X, y):
        self.weights = np.zeros((n_features, 1)); self.bias = 0.0
        for _ in range(self.n_iter):
            y_pred = self._sigmoid(np.dot(X, self.weights) + self.bias)
            self.weights -= self.lr * (1/n) * np.dot(X.T, (y_pred - y))
            self.bias    -= self.lr * (1/n) * np.sum(y_pred - y)
        return self

#
grad = np.vstack([grad_w, [[grad_b]]])
S = np.diag((y_pred * (1 - y_pred)).flatten())
X_aug = np.hstack([X, np.ones((n, 1))])
H = (1/n) * X_aug.T @ S @ X_aug + 1e-5 * np.eye(n_features + 1)
delta = np.linalg.solve(H, grad)
self.weights -= delta[:-1].reshape(-1, 1); self.bias -= delta[-1, 0]
```

## 6.5 Эксперименты

Исследованы гиперпараметры: learning rate $\eta \in \{0.001, 0.01, 0.05, 0.1, 0.5, 1.0\}$, количество итераций $N \in \{100, 500, 1000, 5000, 10000\}$, метод оптимизации (градиентный спуск и метод Ньютона).

Table 5: Влияние learning rate (GD, $N = 5000$)

| $\eta$ | Accuracy | Precision | Recall | F1 |
|---|---|---|---|---|
| 0.001 | 0.7765 | 0.7302 | 0.6667 | 0.6970 |
| 0.01 | 0.7989 | 0.7797 | 0.6667 | 0.7188 |
| 0.05 | 0.8045 | 0.7931 | 0.6667 | 0.7244 |
| **0.1** | **0.8045** | **0.7931** | 0.6667 | **0.7244** |
| 0.5 | 0.8045 | 0.7931 | 0.6667 | 0.7244 |
| 1.0 | 0.8045 | 0.7931 | 0.6667 | 0.7244 |

Table 6: Влияние количества итераций (GD, $\eta = 0.1$)

| $N$ | Accuracy | Precision | Recall | F1 |
|---|---|---|---|---|
| 100 | 0.7989 | 0.7797 | 0.6667 | 0.7188 |
| 500 | 0.7989 | 0.7797 | 0.6667 | 0.7188 |
| 1000 | 0.8045 | 0.7931 | 0.6667 | 0.7244 |
| 5000 | 0.8045 | 0.7931 | 0.6667 | 0.7244 |
| 10000 | **0.8045** | **0.7931** | 0.6667 | **0.7244** |

Table 7: Метод Ньютона — сходимость за 3 итерации

| Итераций | Accuracy | Precision | Recall / F1 |
|---|---|---|---|
| 3 и более | 0.8045 | 0.7931 | 0.6667 / 0.7244 |

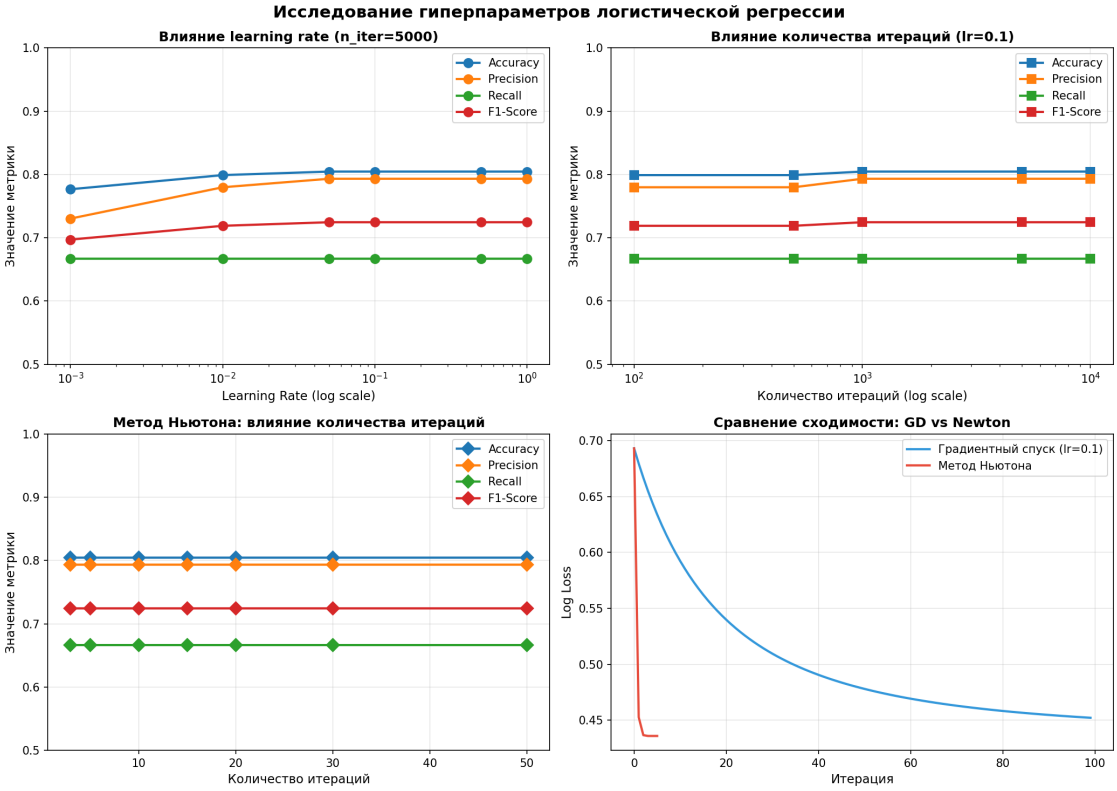



Figure 14: Зависимость метрик от гиперпараметров

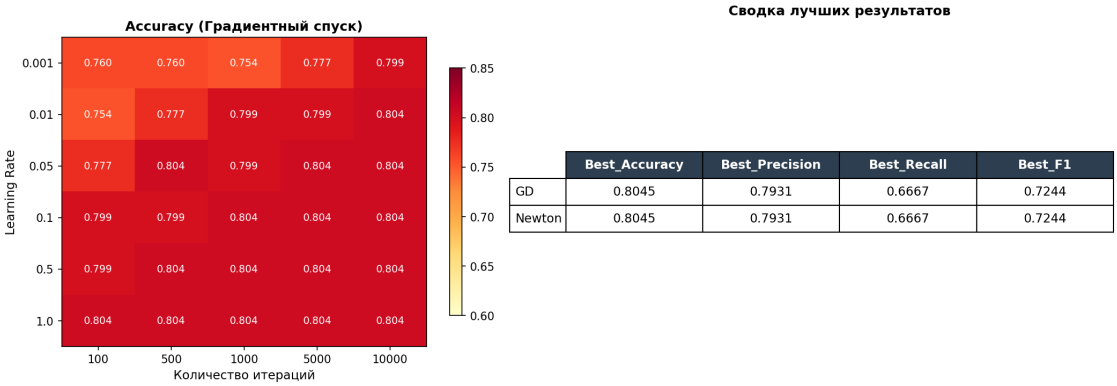



Figure 15: Тепловая карта Accuracy (GD) и сводная таблица

**Анализ:**

- При $\eta = 0.001$ модель сходится медленно (Accuracy $\approx 0.78$). Оптимальный диапазон: $\eta \in [0.01, 0.1]$.
- При $\eta \geq 0.5$ расходимости не происходит благодаря нормализации, но на более сложных датасетах возможна нестабильность.
- После $N = 1000$ итераций наступает плато; дальнейшее обучение не улучшает метрики.
- Метод Ньютона сходится за 3–5 итераций против $\sim 1000$ для GD, но требует $O(d^3)$ на обращение Гессе.
- Precision (0.79) выше Recall (0.67) — модель осторожна с предсказанием класса «выжил», что объясняется дисбалансом классов (61.6% погибших).

## 6.6 Финальная модель

С оптимальными параметрами ($\eta = 0.1$, $N = 10000$, GD):

Table 8: Финальные метрики

| Метрика | Значение | Элемент | Значение |
|---|---|---|---|
| Accuracy | 0.8045 | TN | 98 |
| Precision | 0.7931 | FP | 12 |
| Recall | 0.6667 | FN | 23 |
| F1-Score | 0.7244 | TP | 46 |

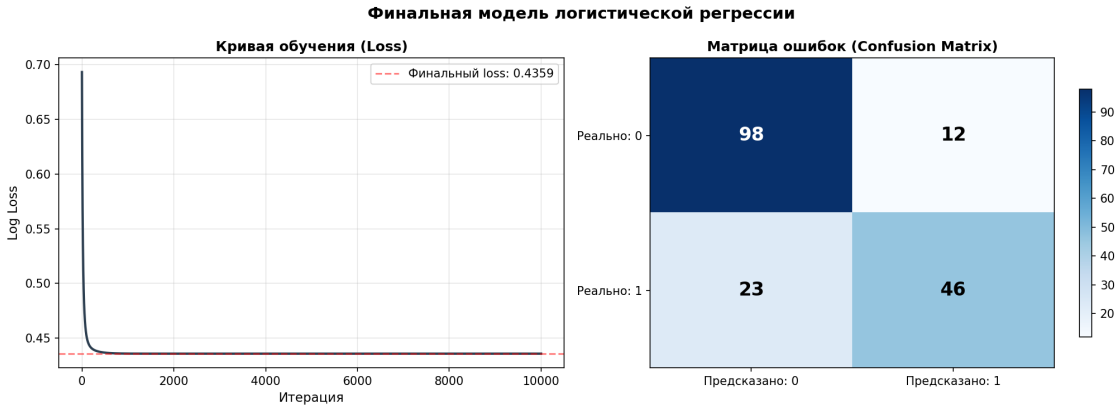



Figure 16: Кривая обучения и матрица ошибок

Модель правильно классифицирует 98 из 110 погибших (Recall $\approx 0.89$), но лишь 46 из 69 выживших (Recall $\approx 0.67$). Это консервативное поведение соответствует априорному распределению классов.

## 6.7 Когда использовать

- Бинарная классификация с требованием вероятностного выхода.
- Задачи, где важна интерпретация коэффициентов (odds ratios).
- Медицинские исследования (вероятность заболевания).
- Базовый классификатор перед переходом к сложным моделям.

# 7 Сравнение методов

## 7.1 Сводная таблица

Table 9: Сравнительная характеристика методов

| Критерий | Лин. регрессия | $k$-NN | Дерево решений | Логист. регрессия |
|---|---|---|---|---|
| Тип задачи | Регрессия | Классификация | Классификация | Классификация |
| Обучение | Аналитическое | Ленивое | Рекурсивное | Итеративное |
| Масштабирование | Желательно | Обязательно | Не требуется | Обязательно |
| Интерпретируемость | Высокая | Низкая | Очень высокая | Высокая |
| Чувств. к выбросам | Высокая | Средняя | Низкая | Средняя |
| Чувств. к размерности | Низкая | Высокая | Средняя | Низкая |
| Нелинейность | Нет | Да | Да | Нет |
| Пропуски | Мешают | Мешают | Обрабатывает | Мешают |
| Скорость predict | $O(d)$ | $O(nd)$ | $O(\text{depth})$ | $O(d)$ |
| Вероятностный выход | Нет | Нет | Да | Да |

## 7.2 Рекомендации «если. . . , то. . . »

1. **Мало признаков + нужна интерпретация** → линейная или логистическая регрессия.
2. **Сложные нелинейные зависимости** → дерево решений.
3. **Нет времени на обучение / быстрый baseline** → $k$-NN.
4. **Категориальные признаки / пропуски в данных** → дерево решений.
5. **Вероятностный прогноз для бинарного исхода** → логистическая регрессия.
6. **Высокая размерность** ($d > 100$) → линейная/логистическая регрессия.
7. **Цена ошибки асимметрична** → метод с настраиваемым порогом + оценка по Precision/Recall.

## 7.3 Итоговые метрики

Table 10: Лучшие результаты модуля

| Метод | Метрика | Значение |
|---|---|---|
| Линейная регрессия | $R^2$ | 0.6446 |
| $k$-NN (Wine) | Accuracy | 0.9815 |
| Дерево решений | ROC-AUC | 0.9828 |
| Логист. регрессия | F1-Score | 0.7244 |

# 8 Заключение

В ходе выполнения Модуля 2 освоены четыре фундаментальных алгоритма машинного обучения, каждый из которых реализован «с нуля» без опоры на высокоуровневые ML-фреймворки. Основные выводы:

1. **Линейная регрессия** — простой и интерпретируемый метод для задач регрессии. Ключевой фактор качества — выбор информативных признаков (`median_income` даёт почти половину объяснённой дисперсии).
2. **$k$-NN** — эффективен при малой размерности и хорошем масштабировании. Качество критически зависит от набора признаков: переход от случайных к информативным повысил Accuracy с 0.93 до 0.98.
3. **Дерево решений** — лучший выбор, когда требуется интерпретируемость. На датасете Mushroom модель глубиной 4 достигла ROC-AUC 0.983, при этом правила легко читаются человеком.
4. **Логистическая регрессия** — надёжный baseline для бинарной классификации с вероятностным выходом. Метод Ньютона сходится на порядки быстрее градиентного спуска, но практический выигрыш проявляется только при больших $d$.
5. **Выбор метода** определяется типом задачи, размером и качеством данных, требованиями к интерпретируемости и допустимой ценой ошибок. Не существует универсального алгоритма — необходимо сопоставлять свойства метода с особенностями конкретной задачи.

## Использование искусственного интеллекта

В ходе работы использовались инструменты ИИ (Claude, ChatGPT) для помощи в написании кода, отладке алгоритмов и оформлении отчёта. Конкретно: реализация метода Ньютона и ID3 с нуля, векторизация расстояний в k-NN, подбор формул для отчёта, генерация графиков и вёрстка LaTeX. Все решения осмыслены и проверены авторами.